

Trade-offs in Partition Assignment Policy intended for Distributed Deployment of Dynamic Graph Embedding Systems

Stefan Nožinić

Seminar 3

September 15, 2026

Outline

- 1 Motivation
- 2 Method
- 3 Experimental Setup
- 4 Results
- 5 Conclusion

Motivation

- Large, dynamic graphs must be embedded in a **distributed** setting: no single machine holds the whole graph.
- Vertices arrive **online** and must be assigned to a partition before their full neighborhood is known.
- Core trade-off:
 - More partitions → better scalability, but can hurt embedding quality and balance.
 - Replicating an uncertain vertex across partitions can recover quality, at extra computation cost.
- This trade-off has not been systematically studied for dynamic graph embedding.

Problem Formulation

Given a dynamic graph with online vertex arrivals and a distributed dynnode2vec-style embedding model, how should partitions be assigned to new vertices to balance:

- embedding quality,
- partition balance,
- computational cost of maintaining the assignment?

Buffered Event-Processing Pipeline

- The graph is a stream of events (t, u, v) .
- Events are **buffered** before partition assignment, so decisions use a batch of neighbor information instead of a single edge.
- Once a buffer fills (`MAX_BUFFER_SIZE`):
 - 1 assign partitions for the buffered vertices,
 - 2 update embeddings,
 - 3 clear the buffer.
- Each buffer B_n applied to snapshot G_n yields the next snapshot: $G_{n+1} = G_n \cup B_n$.

Partition Scoring Formula

Score of partition $p_i \in \mathcal{P}$ for a newly arriving vertex v :

$$S(p_i, v) = N(p_i, v) - \mu \cdot \max \left(0, |p_i| - \alpha(1 + \epsilon) \cdot \frac{1}{P} \sum_{p_j \in \mathcal{P}} |p_j| \right)$$

- $N(p_i, v)$ – neighbors of v already in p_i , within the current buffer
- μ – capacity penalty coefficient
- α – weight of the average partition size
- ϵ – imbalance tolerance
- $|p_i|$ – current size of partition p_i

Vertex v is assigned to the top RF scoring partitions (replication factor RF); ties are broken at random.

Datasets

Dataset	Nodes	Edges	Avg. deg.	Avg. clust.	Density	Modularity
CITeseer	3264	4536	2.78	0.145	0.00090	0.72
DBLP	17716	52867	5.97	0.134	0.00034	0.58
AstroPh	18772	198110	21.11	0.631	0.00112	0.33
AS-Oregon	11806	38781	6.57	0.399	0.00056	0.57
Enron	36692	183831	10.02	0.497	0.00027	0.36

Five real-world dynamic graphs, chosen to vary in size, density, and community structure.

Static Baseline and Hyperparameters

Static node2vec F1 (unpartitioned)

Dataset	F1
CITESEER	34.82%
DBLP	60.3%
AstroPh	70.41%
AS-Oregon	31.26%
Enron	20.42%

Partitioning hyperparameters

Hyperparameter	Value
Buffer size	1000
μ	1
α	1
ϵ	0.1
P	{1, 2, 4, 8}
RF	1

These static F1 scores are a rough upper bound: a partitioned, incremental system generally should not exceed them.

F1 Reconstruction Score vs. Partition Count

Dataset	$P = 1$	$P = 2$	$P = 4$	$P = 8$
CITeseer	47.57%	45.81%	48.06%	54.81%
DBLP	50.91%	54.11%	54.51%	49.87%
AstroPh	57.70%	54.67%	50.95%	45.94%
AS-Oregon	26.17%	24.88%	24.30%	21.45%
Enron	18.03%	24.84%	27.44%	32.36%

- AstroPh, AS-Oregon: quality **degrades** as P grows.
- CITeseer, DBLP, Enron: quality **stable or improves** over most of the range.

Edge Cut and Balance vs. Partition Count

Dataset	Edge cut (%)			
	$P=1$	$P=2$	$P=4$	$P=8$
CITeseer	0	23	39	48
DBLP	0	32	50	59
AstroPh	0	35	56	65
AS-Oregon	0	30.4	53.8	71.6
Enron	0	33.1	49.2	62.0

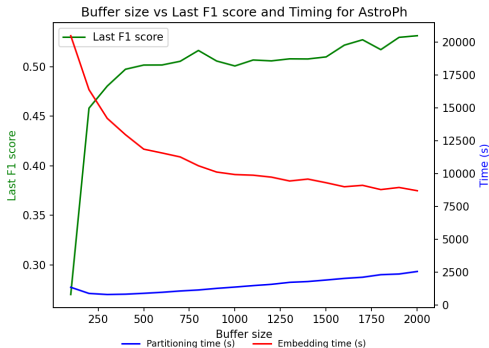
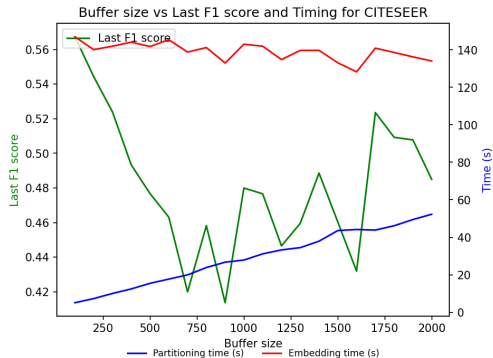
Dataset	Balance ($B=1$ is ideal)			
	$P=1$	$P=2$	$P=4$	$P=8$
CITeseer	1	1.31	1.63	1.75
DBLP	1	1.10	1.20	1.31
AstroPh	1	1.08	1.11	1.26
AS-Oregon	1	1.21	1.26	1.27
Enron	1	1.09	1.08	1.25

Edge cut grows monotonically with P on every dataset; balance stays near 1 for all datasets except CITeseer.

Why the Dataset-Dependent Split?

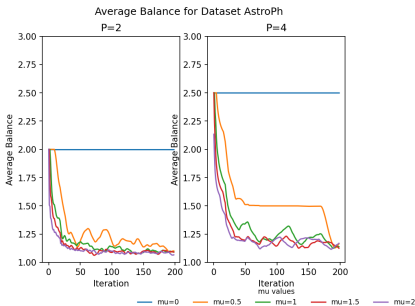
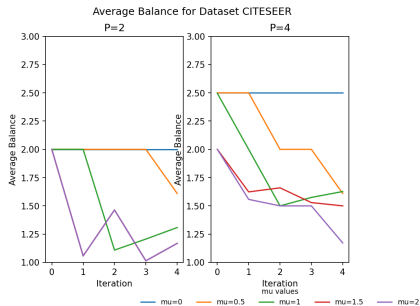
- Signed F1 change $P=1 \rightarrow P=8$ correlates most strongly with graph **density** (Spearman $\rho \approx -0.7$).
- Weaker correlation with modularity ($\rho \approx +0.4$) and clustering coefficient ($\rho \approx -0.3$).
- No single property cleanly separates the two groups:
 - CITESEER (improves) is denser than AS-Oregon (degrades).
 - Enron has the lowest density and modularity among the stable group, yet the largest improvement.
- Density is the closest available proxy, but not a complete explanation.

Buffer Size Sensitivity



- AstroPh: larger buffers clearly help ($F1 \approx 0.27 \rightarrow 0.53$).
- CITESEER: larger buffers do *not* help; best F1 at the smallest buffer size.
- Partitioning time grows with buffer size but stays smaller than embedding time throughout.

Capacity Penalty (μ) Sensitivity



- $\mu = 0$ collapses all vertices into one partition (worst balance: 2.00 / 2.50 at $P=2/4$).
- Increasing μ from 0 to 1 gives the largest balance improvement; returns diminish beyond $\mu \approx 1$ –1.5.
- Better balance is bought at a real F1 cost (e.g. AstroPh $P=8$: 57.00% $\rightarrow$ 33.93% at $\mu=0.5$).

Random Partitioner Baseline

CITESEER

P	F1	Edge cut	Balance
2	0.03%	50.0%	1.02
4	0.03%	70.9%	1.32
8	0.04%	81.6%	1.61

AstroPh

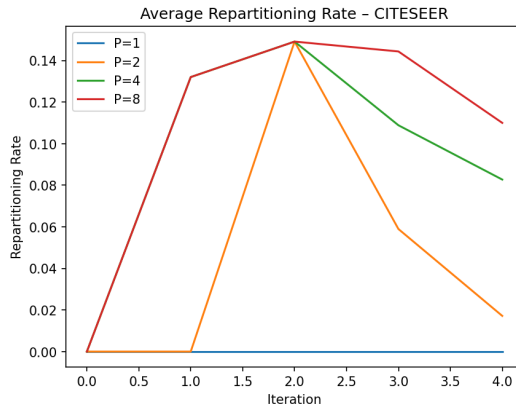
P	F1	Edge cut	Balance
2	0.04%	50.0%	1.00
4	0.03%	70.9%	1.31
8	0.02%	81.4%	1.60

Random assignment collapses F1 to ≈ 0 : the neighbor partitioner's quality genuinely comes from preserving neighborhood locality, not from the embedding algorithm alone.

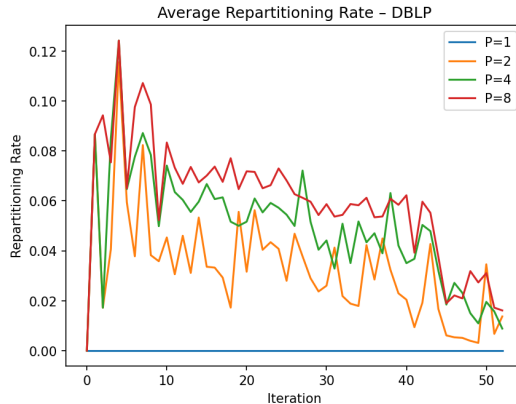
Effect of Replication Factor

- Replication lets a vertex be embedded in more than one partition; the final embedding is the average of the per-partition embeddings.
- Preliminary sweep on CITESEER, $P = 4$, buffer size 1000, $\mu = 1$:
 - $RF = 1$: $F1 = 44.35\%$
 - $RF = 3$: $F1 = 69\%$
- Large recovery of embedding quality lost to early, uncertain assignment – at the cost of up to $RF \times$ the embedding computation per vertex.
- Single-configuration result: promising, but not yet systematically validated.

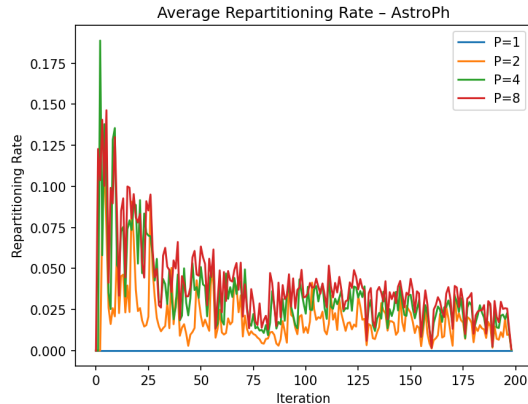
Temporal Evolution: Repartitioning Rate – CITESEER



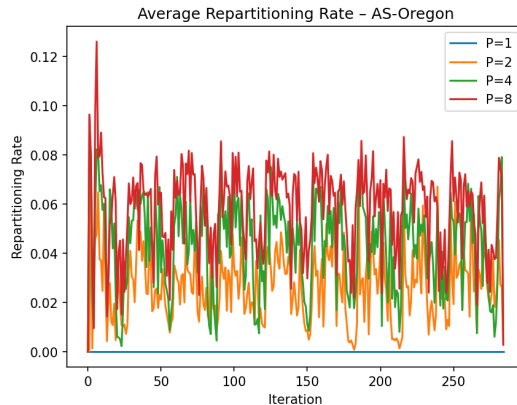
Temporal Evolution: Repartitioning Rate – DBLP



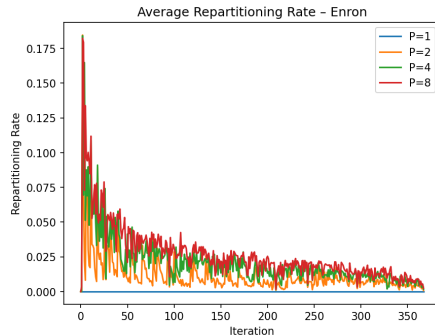
Temporal Evolution: Repartitioning Rate – AstroPh



Temporal Evolution: Repartitioning Rate – AS-Oregon



Temporal Evolution: Repartitioning Rate – Enron



Across all datasets, the repartitioning rate decays rapidly after an initial warm-up and stays below 20% – assignments stabilize quickly.

Conclusion

- Impact of partitioning on embedding quality is **dataset-dependent**: AstroPh and AS-Oregon degrade; CITESEER, DBLP, and Enron are stable or improve, with DBLP reversing sharply at $P = 8$.
- Graph density correlates most closely with this split, but does not fully explain it.
- The neighbor-based partitioner keeps balance close to ideal (except CITESEER) and repartitioning rate low, without explicit rebalancing.
- Partitioning time stays consistently smaller than embedding time.
- Replication factor is a promising, preliminary lever for recovering quality lost to early assignment.

Future Work

- Direct comparison of delayed vs. immediate assignment, measuring latency alongside quality.
- Systematic replication factor sweep across datasets and partition counts.
- Evaluate embeddings on downstream tasks (link prediction, node classification), beyond reconstruction F1.

Thank you!